**Relatório: Gerenciamento das jogadas do osu! com BST e AVL**

Nome: João Maerbal Nunes Barros
RA: 202510288
Disciplina: Algoritmos e Estruturas de Dados · Professor: Elder Vicente de Paulo Sobrinho
Vídeo: https://www.youtube.com/watch?v=bza74MyH1AQ

## 1. Problema e fonte real

O trabalho pede para aplicar árvores BST e AVL a uma situação real em que elas façam diferença e para defender essa escolha. A situação que escolhi é o gerenciamento das jogadas do meu perfil no osu!, um jogo de ritmo que jogo há sete anos. Cada jogada num beatmap recebe uma pontuação chamada pp (Performance Points), e o perfil guarda só a melhor jogada de cada beatmap, identificada pelo ID do mapa.

O programa carrega essas jogadas nas duas árvores e faz as operações que o perfil precisa: buscar, inserir e remover uma jogada pelo ID, além de mostrar os percursos e a altura de cada árvore.

Como extra, incluí a simulação de uma jogada nova. O pp total do perfil é uma soma ponderada, em que a jogada na posição n do ranking vale pp × $0{,}95^{n-1}$, então o programa mostra em que posição uma jogada de x pp entraria e quanto ela valeria. A simulação não depende da organização das árvores, porque percorre todas as jogadas contando quantas têm pp maior, e entra como complemento ao gerenciamento.

Os dados são reais. O arquivo `jogadas_osu.csv` tem as 200 melhores jogadas do meu perfil (osu.ppy.sh/users/13699780), coletadas pelo endpoint público do site, com nome do beatmap, ID do beatmap, pp com três casas decimais e data, em ordem cronológica. A fórmula de ponderação é a da wiki oficial do osu! (osu.ppy.sh/wiki/en/Performance_points). Escolhi esse problema porque são dados meus, que eu entendo e consigo conferir com o que o site mostra.

## 2. Por que árvore

A chave das árvores é o ID do beatmap. Ele identifica a jogada de verdade, porque o perfil guarda uma jogada por beatmap, e uma jogada melhor no mesmo mapa substitui a anterior. O pp não serve como chave porque pode se repetir entre jogadas, então ele é só um dado guardado no nó. As operações principais do programa são buscar, inserir e remover pelo ID, e a árvore é a estrutura que faz as três de forma eficiente:

| Estrutura | Busca por ID | Inserção | Remoção |
|---|---|---|---|
| Vetor ordenado | O(log n), busca binária | O(n), desloca elementos | O(n), desloca elementos |
| Lista encadeada | O(n) | O(1) no início, sem ordem | O(n) para achar o nó |
| BST | O(h): log n em média, n no pior caso | O(h) | O(h) |
| AVL | O(log n) garantido | O(log n) | O(log n) |

A BST resolve o problema do vetor e da lista enquanto a altura h fica baixa, mas depende da ordem de inserção: se os IDs chegam em ordem crescente, ela vira uma lista e h = n. A AVL resolve isso com rotações, que mantêm a diferença de altura entre as subárvores de cada nó em no máximo 1 e garantem altura proporcional a log n em qualquer ordem.

## 3. Experimento comparativo

Com as jogadas inseridas em ordem crescente de ID, a BST degenerou em lista com altura 200, enquanto a AVL ficou com altura 8 e fez quase 15 vezes menos comparações. Os números abaixo são da opção 7 do programa, com as 200 jogadas reais, buscando cada uma delas em cada árvore. Cada cenário tem 200 registros.

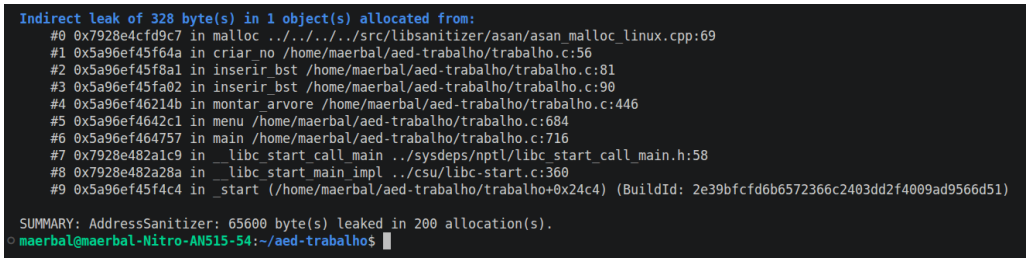
| Cenário | Árvore | Altura | Comparações totais | Média por busca |
|---|---|---|---|---|
| B (crescente) | BST | 200 | 20100 | 100,50 |
| B (crescente) | AVL | 8 | 1353 | 6,76 |
| A (original) | BST | 17 | 1888 | 9,44 |
| A (original) | AVL | 9 | 1368 | 6,84 |

No Cenário B, cada ID inserido é maior que todos os anteriores, então cada nó vai para a direita do último e a BST vira uma lista. Achar a jogada da posição k custa k comparações, e o total é 1 + 2 + … + 200 = 20.100, com média de 100,5. A AVL rotaciona durante as inserções e fica com altura 8, a mínima possível para 200 nós.

No Cenário A, as jogadas entram na ordem do arquivo, que é cronológica. Mapas mais novos tendem a ter ID maior, com correlação de 0,25 entre a ordem por data e o ID, então essa ordem não é aleatória, mas também não é crescente. A BST não degenera, mas fica com altura 17, quase o dobro da AVL. A AVL ficou com altura 8 ou 9 nos dois cenários, o que mostra que ela não depende da ordem de entrada.

## 4. Diário de Bordo

**14/10/2026: teste de vazamento com o AddressSanitizer.** Para confirmar que o AddressSanitizer detecta memória não liberada, comentei de propósito a chamada `liberar_arvore(bst);` no fim do `main`. O relatório do LeakSanitizer tinha milhares de linhas, uma pilha de chamadas por nó perdido, e se misturava com a listagem das jogadas. Separei as saídas com `./trabalho 2>&1 > /dev/null | head -8` e `./trabalho 2>&1 | tail -3`, e o resumo mostrou 67.200 bytes perdidos em 200 alocações, ou seja, 200 nós de 336 bytes alocados em `criar_no`. Ao descomentar a linha e recompilar, o AddressSanitizer não acusou mais nada.




*Final da saída, em reprodução do teste em 18/10, já com a struct atual de 328 bytes por nó: 200 × 328 = 65.600 bytes.*

**14/10/2026: CSV parecia quebrado no LibreOffice.** Ao abrir o CSV no LibreOffice Calc, a linha do mapa Koise yo Minna, Hai! (TV Size) [Insane] apareceu partida em duas colunas, empurrando o ID, o pp e a data para a direita, e os pp apareciam com 2 casas. No terminal, as 200 linhas tinham exatamente 4 campos separados por ponto e vírgula e o pp com 3 casas. O LibreOffice estava usando a vírgula também como separador, e quatro nomes de mapas têm vírgula. Na janela Text Import, deixei marcado só Semicolon.

| | A | B | C | D | E |
|---|---|---|---|---|---|
| 1 | nome_do_mapa | id_beatmap | pp | data | |
| 2 | Shitteru? Madousho wa Donki ni mo Naru no yo [shitteru?] | 1925409 | 351.049 | 2022-10-11 16:20:19 | |
| 3 | Koise yo Minna | Hai! (TV Size) [Insane] | 2054967 | 317.42 | 2022-11-04 21:08:10 |
| 4 | Letters to me [Endless Journey] | 2828520 | 339.707 | 2022-11-24 22:04:21 | |
| 5 | Howling (TV Size) [Sotarks' Exodus] | 2161155 | 313.87 | 2022-12-27 21:34:38 | |
| 6 | Caffeine Fighter [Kowari's Insane] | 2188430 | 354.056 | 2023-01-27 18:28:25 | |
| 7 | Rinne [Eternal feeling] | 3501824 | 319.026 | 2023-02-04 18:19:24 | |
| 8 | Heart Realize [Eternity] | 3014649 | 307.461 | 2023-02-17 01:06:29 | |
| 9 | FaV (Short Ver.) [Kowari's Extra] | 2559127 | 309.897 | 2023-02-18 16:40:44 | |
| 10 | Not For Sale Fossil [Frail] | 2389623 | 343.088 | 2023-02-25 18:48:15 | |

**14/10/2026: última linha da saída sumia no terminal do VS Code.** Ao buscar um ID inexistente, a saída terminava em `ID 1 nao encontrado.` sem a linha `Comparacoes: 6`, embora o `printf` estivesse fora do `if/else`. Redirecionei a saída para um arquivo com `./trabalho > saida.txt`, e o arquivo tinha as 5 linhas esperadas. O programa estava certo; o terminal tinha deixado de mostrar a linha ao redesenhar a tela, e rodar `clear` antes de executar resolveu.

**18/10/2026: função usada antes de ser declarada.** Ao adicionar a função `testar_percursos`, o compilador avisou `implicit declaration of function 'liberar_arvore'` e depois `conflicting types for 'liberar_arvore'`. A nova função estava no arquivo antes da `liberar_arvore`, e o C, que lê de cima para baixo, ainda não conhecia a função no ponto da chamada. Movi a `testar_percursos` para depois da `liberar_arvore` e o programa voltou a compilar sem avisos.

**21/10/2026: cabeçalho da função comentado no lugar da chamada.** Para reproduzir o teste de vazamento, a ideia era comentar a chamada `liberar_arvore(bst);`, mas comentei a linha 457, que é o cabeçalho da própria função. O compilador acusou `expected identifier or '(' before '{' token` na linha 458, porque o corpo da função ficou solto no arquivo, e depois avisou `implicit declaration of function 'liberar_arvore'` na linha 523, porque a função deixou de existir. Descomentei o cabeçalho e comentei só a chamada que libera a BST montada pelo menu, e o programa voltou a compilar e a mostrar o vazamento esperado.

